

SMC Scratchpad RAM — Low-Level Design

SMC Scratchpad RAM — Low-Level Design

Document: 02_SCRATCHPAD_RAM_LowLevel_Design.md **Module:** smc::scratchpad_ram **Companion docs:** 01_SCRATCHPAD_RAM_Specification.md (external contract), 03_SCRATCHPAD_RAM_Test_Plan.md (verification)

Contents

1. Overview
 2. File layout
 3. Class structure
 - 3.1 Ports
 - 3.2 Public methods (API)
 - 3.3 Internal methods and SC processes
 - 3.4 File-local helpers
 - 3.5 Function call flow
 4. CCI parameter scaffolding
 5. Construction sequence
 6. b_transport data path
 7. Byte-enable handling
 8. SECDED ECC implementation
 9. t_transport_dbg and the debug API
 10. Preload pipeline
 11. Reset handling
 12. Error taxonomy
 13. Relationship to the Boot ROM model
 14. Build & packaging
 15. Performance notes
 16. Future work
-

1. Overview

smc::scratchpad_ram is a Loosely-Timed TLM-2.0 target modelling the SMC CPU cluster's on-chip SECDED scratchpad. It is the **read-write sibling** of smc::bootrom: it shares the same access-validation logic, preload pipeline, CCI scaffolding, and error taxonomy, and adds (a) committing writes, (b) byte-enable handling, and (c) a behavioural SECDED ECC model.

Design priorities, in order: **functional fidelity to the RTL contract**, **determinism**, and **consistency with the sister SMC IPs**.

2. File layout

```
scratchpad_ram/
├── include/
│   ├── scratchpad_ram.h          SC_MODULE declaration + cci_param + cfg struct
│   └── smc_tlm_extensions.h      Shared smc_axi_extension (copied from the suite)
├── src/
│   └── scratchpad_ram.cpp        Implementation
└── test/
    ├── scratchpad_ram_tb.cpp     Primary self-checking bench
    ├── scratchpad_ram_neg_tb.cpp Negative-path / edge-case bench
    └── fixtures/scratchpad_sanity.rv64.hex
```

3. Class structure

```
class scratchpad_ram : public sc_core::sc_module {
    // CCI params (constructed first; declared before the public ports)
    cci::cci_param<uint64_t, CCI_IMMUTABLE_PARAM> size_bytes_p_;
    cci::cci_param<std::string, CCI_IMMUTABLE_PARAM> init_file_p_;
    cci::cci_param<std::string, CCI_IMMUTABLE_PARAM> init_file_format_p_;
    cci::cci_param<double> access_delay_ns_p_; // mutable
    cci::cci_param<bool, CCI_IMMUTABLE_PARAM> ecc_enabled_p_;
public:
    tlm_utils::simple_target_socket<scratchpad_ram> reg_socket;
    sc_core::sc_in<bool> rst_n_i;
    // debug API: dbg_read64/32, dbg_load_bytes, dbg_inject_ecc_error,
    //             dbg_clear_ecc_errors, size_bytes, ecc_enabled, dump_state
private:
    scratchpad_ram_cfg cfg_;
    std::vector<uint8_t> data_; // size = cfg_.size_bytes
    sc_core::sc_time access_delay_;
    std::unordered_map<uint64_t, bool> ecc_errors_; // word offset → uncorrectable?
};
```

`data_` is a single flat byte vector. The RTL splits the 64 KiB array into 4 partitions, but that is a physical-implementation detail invisible at the transaction level — a flat array is functionally identical.

3.1 Ports

Port	Type	Dir	Purpose
reg_socket	tlm_utils::simple_target_socket<scratchpad_ram>	target	TLM-2.0 read/write access (AXI / AXI-Lite / TileLink-style initiator).
rst_n_i	sc_core::sc_in<bool>	in	Active-low reset. No-op for SRAM contents; kept for IP-suite symmetry.

3.2 Public methods (API)

Method	Signature	Purpose
constructor	scratchpad_ram(sc_module_name, scratchpad_ram_cfg = {})	Build, validate config, allocate + optionally preload, register callbacks/process (see §5).
dbg_read64	uint64_t dbg_read64(uint64_t off) const	Back-door aligned 64-bit read; returns 0 if OOB/unaligned. ECC-bypassing, no delay.
dbg_read32	uint32_t dbg_read32(uint64_t off) const	Back-door aligned 32-bit read; returns 0 if OOB/unaligned.
dbg_load_bytes	unsigned dbg_load_bytes(uint64_t off, const uint8_t* ptr, unsigned len)	Commit len bytes into data_ ; clears ECC flags on touched words; returns bytes written (0 if OOB).
dbg_inject_ecc_error	void dbg_inject_ecc_error(uint64_t off, bool correctable)	Inject a SECEDED error on the covering word (no-op if ecc_enabled == false or OOB).
dbg_clear_ecc_errors	void dbg_clear_ecc_errors()	Drop all injected ECC error flags.
size_bytes	uint64_t size_bytes() const	CCI-resolved total RAM size.
ecc_enabled	bool ecc_enabled() const	CCI-resolved ECC-enable flag.
dump_state	void dump_state(std::ostream& = std::cout) const	Human-readable config + first 32 bytes of contents.

3.3 Internal methods and SC processes

Member	Signature	Kind	Purpose
<code>b_transport</code>	<code>void b_transport(tlm::tlm_generic_payload&, sc_core::sc_time&)</code>	TLM <code>b_transport</code> callback	The blocking read/write data path (§6).
<code>transport_dbg</code>	<code>unsigned int transport_dbg(tlm::tlm_generic_payload&)</code>	TLM <code>transport_dbg</code> callback	Zero-delay, ECC-bypassing back-door read/write (§9).
<code>reset_proc</code>	<code>void reset_proc()</code>	SC_METHOD (sensitive to <code>rst_n_i</code> , <code>dont_initialize()</code>)	Reset handler; empty body — SRAM retains contents (§11).
<code>ecc_check</code>	<code>int ecc_check(uint64_t off, unsigned len, bool scrub)</code>	private helper	Classify ECC status over <code>[off, off+len)</code> : 0 clean / 1 correctable (scrub on read) / 2 uncorrectable (§8).
<code>ecc_clear_range</code>	<code>void ecc_clear_range(uint64_t off, unsigned len)</code>	private helper	Drop ECC flags on covered words after a write re-encodes them (§8).
<code>resolve_format</code>	<code>static std::string resolve_format(const std::string& format, const std::string& path)</code>	static helper	Resolve "auto" to "hex" / "bin" from the filename suffix (§10).
<code>load_preload</code>	<code>void load_preload(std::vector<uint8_t>& const)</code>	private helper	Parse the preload image into <code>data_</code> ; <code>SC_REPORT_FATAL</code> on error (§10).
<code>parse_hex_line</code>	<code>static bool parse_hex_line(const std::string& line, uint64_t& value, bool& consumed)</code>	static helper	Parse one hex preload line; <code>consumed=false</code> for blank/comment lines (§10).

Processes/threads: the model registers exactly **one** SC process — `reset_proc` (`SC_METHOD`). There are **no** `SC_THREAD` / `SC_CTHREAD` processes: all bus activity is driven synchronously through the `b_transport` / `transport_dbg` socket callbacks.

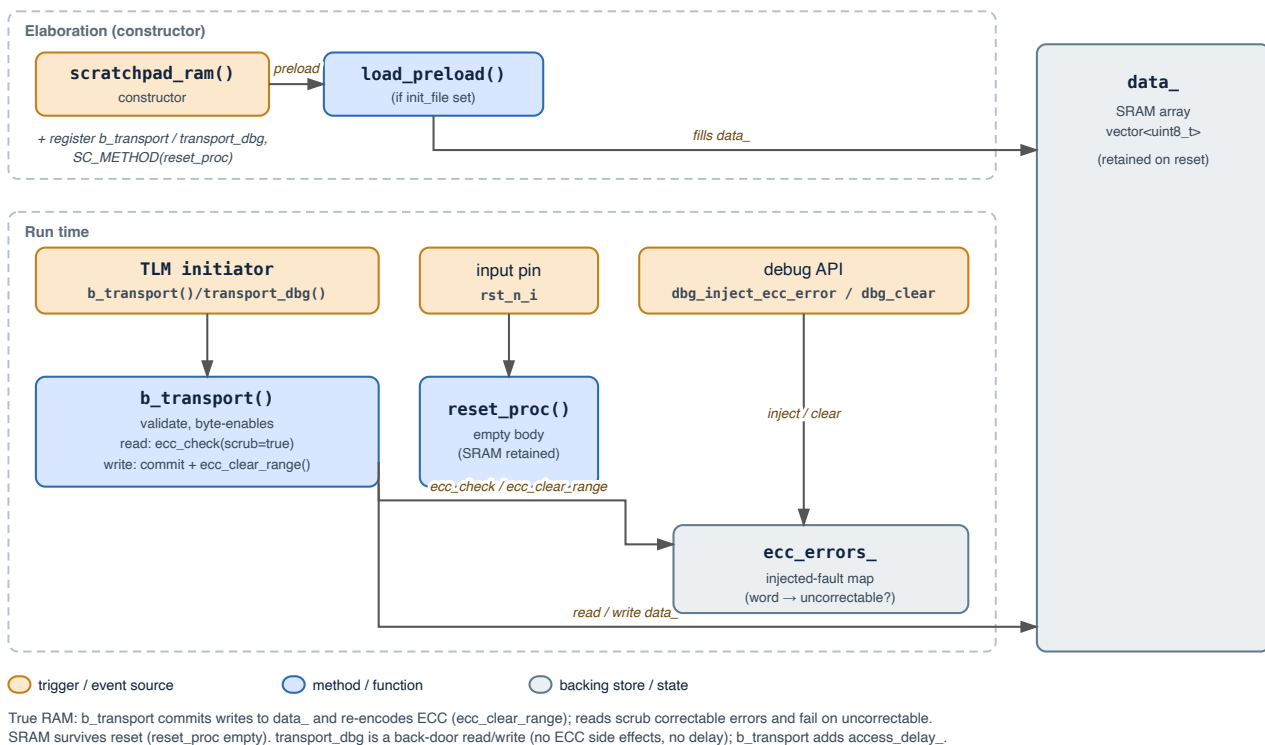
3.4 File-local helpers (`scratchpad_ram.cpp` anonymous namespace)

Function	Signature	Purpose
<code>to_lower</code>	<code>std::string to_lower(std::string s)</code>	Lowercase copy for case-insensitive matching.
<code>ends_with_ci</code>	<code>bool ends_with_ci(const std::string& s, const std::string& suffix)</code>	Case-insensitive suffix test; used by <code>resolve_format</code> to spot <code>.img / .bin</code> .
<code>is_supported_width</code>	<code>constexpr bool is_supported_width(unsigned len)</code>	True for <code>len ∈ {1,2,4,8}</code> .
<code>is_aligned</code>	<code>constexpr bool is_aligned(uint64_t addr, unsigned len)</code>	True iff <code>addr</code> is naturally aligned to <code>len</code> .

3.5 Function call flow

Like the Boot ROM, the scratchpad RAM has **no output ports and no recompute spine** — it is a passive TLM target over a `data_` SRAM array. Unlike the ROM it is **read/write** and adds a SECDED **ECC fault map** (`ecc_errors_`). The diagram below shows elaboration (the constructor fills `data_`) and run time (reads/writes against `data_` plus the ECC side-channel).

Scratchpad RAM — SystemC function-call flow



Scratchpad RAM SystemC function-call flow: the constructor preloads the `data_` SRAM array; `b_transport` reads/writes `data_` with SECDED ECC (`ecc_errors_` fault map); `reset_proc` retains contents on reset.

Reading the flow:

- Elaboration** — `scratchpad_ram()` → (if `init_file` is set) `load_preload()` → fills `data_`. The constructor also registers the TLM callbacks and `SC_METHOD(reset_proc)`.

2. **Read/write** — TLM initiator → `b_transport()` validates and honours byte-enables, then reads or writes `data_`. A read calls `ecc_check(scrub=true)` (uncorrectable → `TLM_GENERIC_ERROR`, correctable → repaired/scrubbed); a write commits to `data_` and calls `ecc_clear_range()` to re-encode ECC. Both touch the `ecc_errors_` fault map. `delay += access_delay_`.
 3. **Back-door / debug** — `transport_dbg()` is a zero-delay read/write of `data_` with **no ECC side effects**; `dbg_inject_ecc_error()` / `dbg_clear_ecc_errors()` mutate `ecc_errors_` directly.
 4. **Reset** — `reset_proc()` (`SC_METHOD` on `rst_n_i`) has an empty body; SRAM contents survive reset (only RTL bus pipeline registers reset).
-

4. CCI parameter scaffolding

The five parameters follow the SMC convention: declared **before** the public ports so they are constructed first (the constructor reads `size_bytes` to size `data_`). The `cfg_` struct fields supply the **defaults**; broker presets override them. After construction the resolved values are copied back into `cfg_` so the rest of the constructor and the debug API see consistent data.

Each parameter carries provenance metadata (`rtl_source`, `valid_values`, `unit`, `ecc_code`, ...) for introspection tools.

5. Construction sequence

1. Construct CCI params (defaults from `cfg`).
2. Copy resolved CCI values into `cfg_`.
3. Attach metadata.
4. Validate: `size_bytes != 0`, `size_bytes % 8 == 0`, `access_delay_ns >= 0` → `SC_REPORT_FATAL` on violation.
5. `data_.assign(size_bytes, 0);` if `init_file` non-empty, `load_preload()`.
6. Cache `access_delay_`.
7. Register `b_transport` / `transport_dbg`.
8. Register `reset_proc` (`SC_METHOD` on `rst_n_i`, `dont_initialize()`).
9. Emit an `SC_REPORT_INFO` banner with the resolved configuration.

The constructor below realises that sequence (CCI description strings trimmed to `"..."` for readability; see `src/scratchpad_ram.cpp` for the full text):

```

scratchpad_ram::scratchpad_ram(sc_core::sc_module_name name,
                               scratchpad_ram_cfg cfg)
    : sc_core::sc_module(name)
    // (1) CCI params constructed first (declared before the public ports).
    //     cfg.* values are the DEFAULTS; broker presets override them.
    , size_bytes_p_      ("size_bytes",      cfg.size_bytes,      "...")
    , init_file_p_       ("init_file",       cfg.init_file,       "...")
    , init_file_format_p_ ("init_file_format", cfg.init_file_format, "...")
    , access_delay_ns_p_ ("access_delay_ns",  cfg.access_delay_ns, "...")
    , ecc_enabled_p_     ("ecc_enabled",     cfg.ecc_enabled,     "...")
    , reg_socket("reg_socket")
    , rst_n_i   ("rst_n_i")
    , cfg_(cfg)
{
    // (2) Sync cfg_ with the (possibly preset-overridden) CCI values so the
    //     rest of the constructor sees consistent data.
    cfg_.size_bytes      = size_bytes_p_.get_value();
    cfg_.init_file       = init_file_p_.get_value();
    cfg_.init_file_format = init_file_format_p_.get_value();
    cfg_.access_delay_ns = access_delay_ns_p_.get_value();
    cfg_.ecc_enabled     = ecc_enabled_p_.get_value();

    // (3) Provenance metadata for introspection tools.
    size_bytes_p_.add_metadata("rtl_source",
                              cci::cci_value(std::string("WithScratchpadWithECC(size=0x10000)")));
    init_file_format_p_.add_metadata("valid_values",
                                     cci::cci_value(std::string("hex|bin|auto")));
    ecc_enabled_p_.add_metadata("ecc_code", cci::cci_value(std::string("SECDED")));
    // ... (further default_KiB / unit / tlm_phase metadata) ...

    // (4) Validate – fail loud at elaboration, not at first transaction.
    if (cfg_.size_bytes == 0)
        SC_REPORT_FATAL(name, "scratchpad_ram size_bytes must be non-zero");
    if ((cfg_.size_bytes % scratchpad_ram_cfg::WORD_BYTES) != 0)
        SC_REPORT_FATAL(name,
                        "scratchpad_ram size_bytes must be a multiple of 8 (native word width)");
    if (cfg_.access_delay_ns < 0.0)
        SC_REPORT_FATAL(name, "scratchpad_ram access_delay_ns must be >= 0");

    // (5) Allocate (zero-filled = cold-boot SRAM) and optionally preload.
    data_.assign(cfg_.size_bytes, 0);
    if (!cfg_.init_file.empty())
        load_preload(data_); // SC_REPORT_FATAL on error

    // (6) Cache the time-typed delay; re-cached on every b_transport so a
    //     CCI mutation takes effect on the next access.
    access_delay_ = sc_core::sc_time(cfg_.access_delay_ns, sc_core::SC_NS);

    // (7) Register the TLM target-socket callbacks.
    reg_socket.register_b_transport (this, &scratchpad_ram::b_transport);
    reg_socket.register_transport_dbg(this, &scratchpad_ram::transport_dbg);

    // (8) Reset method (logging-only; SRAM retains its contents).
    SC_METHOD(reset_proc);
    sensitive << rst_n_i;
    dont_initialize();

    // (9) Human-readable "instantiated with ..." banner.
    SC_REPORT_INFO(name, /* size_bytes / init_file / format / delay / ecc */ "...");
}

```

As in the Boot ROM, the ordering is load-bearing: CCI values must be resolved (2) before validation (4), the size must be known before allocation (5), and the callbacks must be registered (7) before elaboration ends.

6. `b_transport` data path

```
b_transport(gp, delay):
    width check      → TLM_BURST_ERROR_RESPONSE
    alignment check  → TLM_BURST_ERROR_RESPONSE
    streaming check  → TLM_BURST_ERROR_RESPONSE
    window check     → TLM_ADDRESS_ERROR_RESPONSE
    if read:
        ecc_check(scrub=true) == uncorrectable → TLM_GENERIC_ERROR_RESPONSE
        copy data→buf (byte-enable aware)
    elif write:
        copy buf→data_ (byte-enable aware)
        ecc_clear_range()                # re-encode touched words
    else:
        TLM_COMMAND_ERROR_RESPONSE
    delay += access_delay_ns ; status = OK ; dmi_allowed = false
```

Error-precedence order (width → alignment → streaming → window → ECC) matches the Boot ROM, so the two models report identical errors for identical bad transactions.

7. Byte-enable handling

When `byte_enable_ptr != nullptr` and `byte_enable_length != 0`, the model iterates byte-by-byte and applies `byte_enable[i % be_len]`:

- **Read:** only enabled bytes are copied into the caller's buffer; disabled byte positions are left untouched.
- **Write:** only enabled bytes are written into `data_`; disabled bytes keep their previous value.

This mirrors the RTL's 8-bit byte mask. The `be_len` modulo supports the TLM convention of a repeating byte-enable pattern shorter than the data length.

8. SECDED ECC implementation

ECC errors are tracked in a sparse `std::unordered_map<uint64_t, bool>` keyed by **word-aligned** byte offset; the value is the *uncorrectable* flag.

- `ecc_check(off, len, scrub)` walks every word overlapping `[off, off+len)`:
 - returns `2` immediately if any covered word is flagged uncorrectable;
 - otherwise returns `1` if any covered word is flagged correctable, and (when `scrub == true`, i.e. on a read) erases that flag — modelling SECDED's in-place correction on access;
 - returns `0` when clean. Short-circuits to `0` when the map is empty or `ecc_enabled == false`, so there is zero overhead in the common case.
- `ecc_clear_range(off, len)` drops the flags on every covered word; called after a write (the RMW re-encodes fresh ECC).

A read that hits an uncorrectable word returns `TLM_GENERIC_ERROR_RESPONSE`, the LT analogue of the TileLink `d_corrupt` bit.

9. `transport_dbg` and the debug API

`transport_dbg` shares the validation path with `b_transport` but is **read-write, zero-delay**, and **ECC-bypassing** (a debugger sees raw memory and is not subject to ECC faults). Debug writes still call `ecc_clear_range()`.

The debug back-door API (no socket, no delay):

Method	Purpose
<code>dbg_read64(off) / dbg_read32(off)</code>	Aligned word read; returns 0 if OOB/unaligned.
<code>dbg_load_bytes(off, ptr, len)</code>	Commit bytes; clears ECC flags on touched words; returns bytes written.
<code>dbg_inject_ecc_error(off, correctable)</code>	Inject a SECEDED error on the covering word.
<code>dbg_clear_ecc_errors()</code>	Drop all injected error flags.
<code>dump_state(os)</code>	Human-readable config + first 32 bytes.

10. Preload pipeline

Identical to the Boot ROM: `resolve_format()` → `load_preload()` → `parse_hex_line()`. Hex is one 64-bit big-endian ASCII word per line (blank / `#`-comment lines skipped, optional `0x/0X` prefix); binary is a raw little-endian stream. Overflow past `size_bytes` is tolerated only if every overflow word is zero. All preload errors are `SC_REPORT_FATAL` at elaboration.

11. Reset handling

`reset_proc` is an `SC_METHOD` sensitive to `rst_n_i`, registered with `dont_initialize()`. Its body is intentionally empty: SRAM retains its contents across a logical reset, and the RTL only resets the bus-pipeline registers. The method exists for IP-suite symmetry and as an extension point.

12. Error taxonomy

Layer	Mechanism	Used for
Elaboration	<code>SC_REPORT_FATAL</code>	Misconfiguration (size, delay, preload).
Runtime bus	TLM response codes	Bad width/alignment/window/command, uncorrectable ECC.
Debug	Return value 0	<code>transport_dbg</code> validation failure.

13. Relationship to the Boot ROM model

The scratchpad and Boot ROM are the read-write / read-only ends of the same generic `mem` block (both are 64-bit `mem` RDLs differing only in `sw` / `hw` access and size). The models deliberately share structure:

Aspect	Boot ROM	Scratchpad RAM
Writes	Silently ignored (<code>TLM_OK</code>)	Commit to <code>data_</code>
Byte-enables	Rejected (<code>TLM_BYTE_ENABLE_ERROR</code>)	Honoured (per-byte mask)
ECC	None	Behavioural SECCDED
Default delay	1 ns	2 ns (pipelined)
Reset	No-op (no state)	No-op (SRAM retains)
Preload / CCI / errors	Shared design	Shared design

A future refactor could hoist the shared access-validation and preload code into a common `smc::detail::mem_base`, with ROM/RAM/ECC behaviour as policy.

14. Build & packaging

`CMakeLists.txt` builds `libsmc_scratchpad_ram.a` and two CTest benches. SystemC is discovered via `SystemC::systemc` (CMake config) or `SYSTEMC_HOME`; CCI via `CCI_HOME`. `run_tests.sh` wraps `configure/build/run` with `--clean`, `--ctest`, `--asan`, and `--coverage` modes (isolated build dirs for the instrumented variants). The C++ standard is matched to the linked SystemC via `smc_detect_systemc_cxx_std()`.

15. Performance notes

- Reads/writes are an `O(len)` `memcpy` (or byte loop under byte-enables).
- The ECC map is empty in normal operation, so `ecc_check` short-circuits in `O(1)`.
- `access_delay_ns` is re-read from CCI on every transaction (two adds) so run-time mutation takes effect immediately.

16. Future work

1. **DMI fast-path** — grant read-only DMI (and gate write-DMI behind an ECC-bypass acknowledgement) for high-throughput scenarios.
2. **Atomic memory operations** — model the TileLink arithmetic/logical AMOs that the RTL implements via its RMW ALU.
3. **Bit-accurate SECCDED** — optional true Hamming code with per-bit fault injection, for ECC-DV scenarios.
4. **Shared `mem_base`** — factor the common ROM/RAM access path into a base class.

End of document.